IIT MADRAS
Indian Institute of Technology Madras

CS6886 · COURSE PROJECT FINALIZED PROPOSAL · OCT 2026 – with final changes and modifications

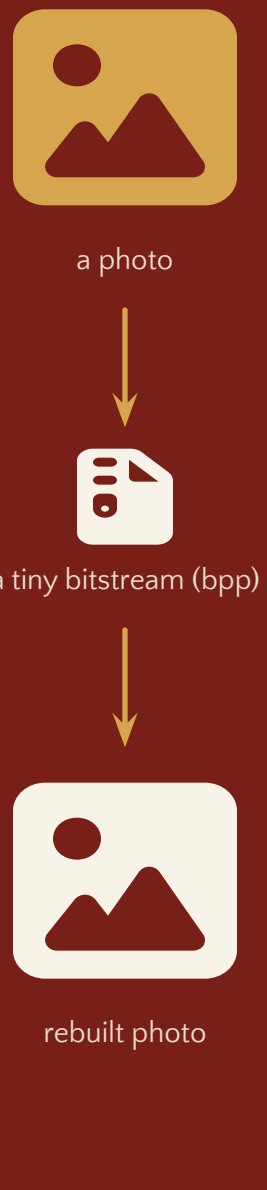
Making PULSE more reliable and usable on lower-end HW

Reproducing PULSE-MSE, mapping FP16/INT8 sensitivity across 8 QPs, recovering quality with QAT, and optimizing mixed precision with ILP

Surya Harshith Balla – CS23B010

CS6886 · Indian Institute of Technology Madras

Paper under test: Jia et al., PULSE — arXiv:2609.18602



Problems with neural codecs — and PULSE's fixes

Codec = coder + decoder

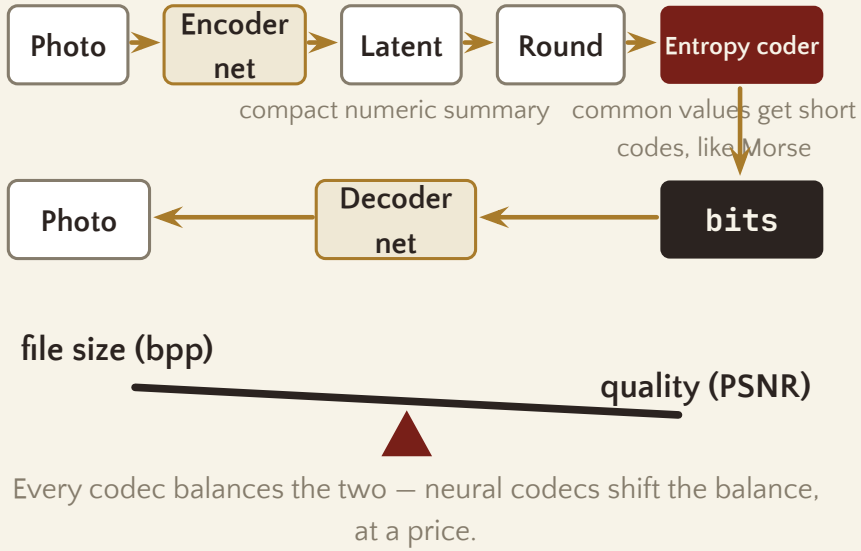
One side shrinks, the other rebuilds. We already use this every day: WhatsApp photos, iPhone HEIC, YouTube video.

Always a trade-off

File size (**bpp**, bits per pixel) vs. quality (**PSNR**, in dB — higher is better).

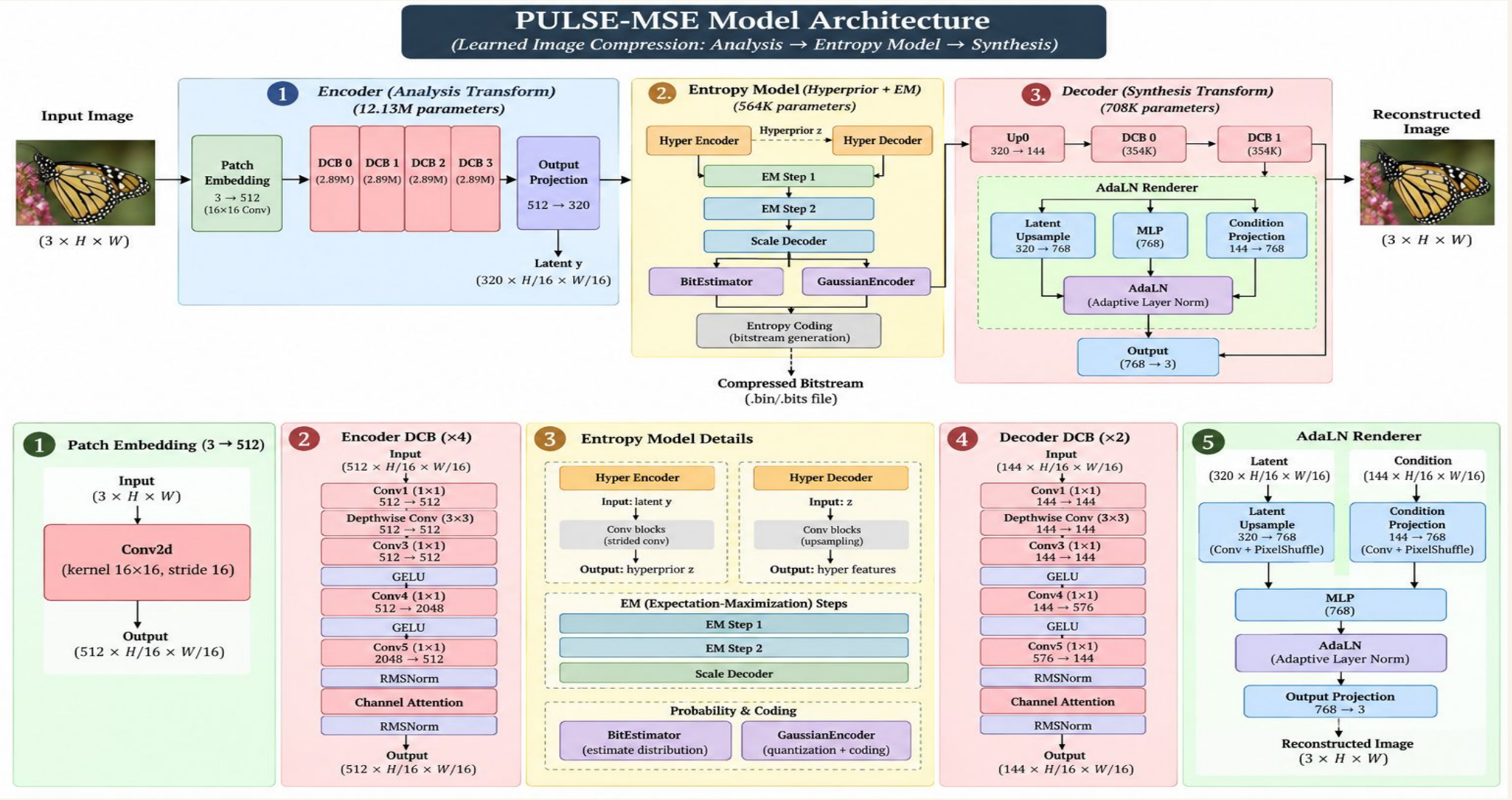
PROBLEM	PULSE'S FIX
Decoder too heavy for phones and CPUs	**Heavy sender, tiny receiver — put the muscle on the encoding side**
8 quality settings used to mean 8 separate models	**One model, 8 presets — QP 0–7** repo README: 0–7 runs from lower to higher rate & quality
Chips round floats differently — a mismatched probability garbles the whole file	**Integer math exactly where both ends must agree — bit-exact entropy coding**

A neural codec's pipeline



Every codec balances the two — neural codecs shift the balance, at a price.

PULSE architecture - considering their base model pulse-mse



How PULSE was structured and tested (taking pulse-mse)

ENCODER · 12.13M params
Patch Embed 3→512
4 × DCB (512-channel)
Output Projection 512→320

ENTROPY MODEL · 1.34M params
Hyper Encoder / Decoder
EM Step 1 · EM Step 2
Scale Decoder + integer entropy control

DECODER · 0.71M params
320→144 projection
2 × DCB
AdaLN renderer

Hardware

Speed

· 1 thread of an AMD EPYC 9V84 server CPU

· Snapdragon 8 Elite phone NPU (REDMI K80 Pro)

Bit-exactness

· x86 CPU · ARM CPU · phone NPU · A100 / H100 GPUs

Training

· One NVIDIA H100: 15 h (MSE), 36 h (perceptual); 22 training runs for the whole architecture search

Benchmarks & metrics

Data

· Kodak — 24 classic test photos, 768×512

· CLIC 2020 Professional validation — 41 high-res photos

Metrics

· PSNR & MS-SSIM vs. bpp at all 8 settings (rate–distortion curve)

· BD-rate vs. 8 separate models (extra % of bits at equal quality; 0% = equal)

· Latency at 1080p · stream-hash match across devices

Reported results

· 1080p decode in **77 / 126 / 420 ms** (S / Base / L) on one CPU thread

· Snapdragon NPU: **42 ms** encode / **20 ms** decode at 1080p

· **Identical stream hashes** across phone, laptop and server — 0 mismatches

· Matches HM on CLIC; −22.5% FID BD-rate vs. MS-ILLM (PULSE-L, perceptual)

Training data (paper, App. B): 4.8M crops from SAM-1B + Open Images at 512^2, then 4,035 high-res images at 1024^2 (800 DIV2K + 2,650 Flickr2K + 585 CLIC train). **Training code is released** — train_mse.py and recipes ship in the repo.

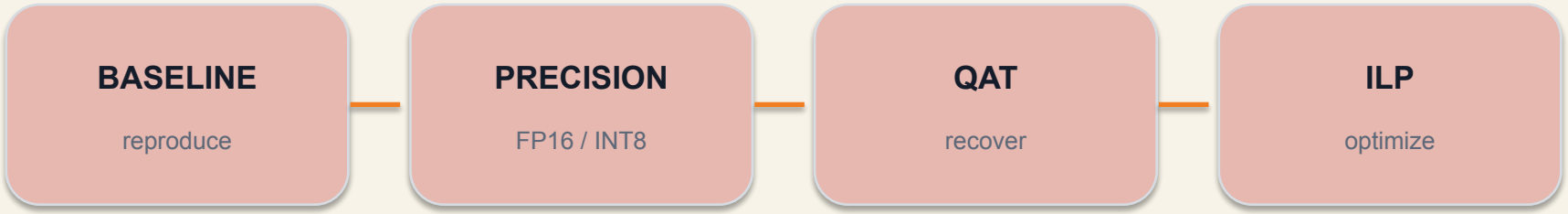
THE PAPER → OUR PLAN

The study we want to extend

The gap: laptop-GPU speed · 8-bit math in the big networks (FP16/BF16 today) · whether the quality setting changes 8-bit behavior · energy · where the time goes.

> **How does one INT8 configuration work across all 8 quality settings, how quantization sensitivity depend on QP and model component ?**

So here we follow these 4 steps in order to reduce the risk on single approach



METRICS WE USE

bpp | PSNR / ΔPSNR | per component latency | memory | energy | correctness

We're **not** claiming PULSE needs a GPU — the RTX 3050 is our measuring instrument; a realistic use is batch-encoding photos on a PC. And these are hunches, not claims: if both are wrong, that's still a result.

Deliverables at the end

D1 Reproduce and measure

PULSE reproduced on the 3050 at all 8 settings, As specified in their artifacts

Time Analysis breakdown(per each significant step done in the model running) and ensuring all the metrics are working.

D2 8-bit across quality settings

TensorRT FP16 / INT8 receiver engines, with a **shared-vs-per-QP BD-rate table**. - if any similar pattern found across a row or column then we have to look into it through D3.

D3 QAT - Quantization Aware Training (from the training sets they have mentioned)

We do this only if we are having very high loss on int8 PTQ at some particular components.

also we add extra testable image sets inorder to make the model bias towards test image sets. - we use tensorRT, PyTorch

D4 Finally - Standard ILP based mixed precision and then publishing the finalized model

We use Integer Linear programming to determine under which constraints and conditions of components - we are able to get the best results

we mainly focus on FP16, INT8 - we use Gurobi or Google OR-Tools (the approach will be based on HAWQ-V3 or custom greedy based algorithm)

we use variables - Latency, PSNR, Memory, Compute(BOPS/ kMAC/pxl), Energy

Baseline metrics [locally on my laptop]

Purpose: Establish a reference for comparing FP32 performance with quantized models.

Energy: We sample GPU power (W) using `nvidia-smi` every 200 ms and estimate total energy by trapezoidal integration: [time-power product summation over intervals], giving energy in joules (J).

Memory: We track total GPU memory usage (MiB) reported by `nvidia-smi` throughout the run, recording the peak and average sampled usage; this is GPU-wide memory, not memory allocated exclusively to the model.

on clic_professional

https://data.vision.ee.ethz.ch/cvl/clic/professional_valid_2020.zip

- **Dataset:** 41 CLIC Professional images evaluated at 8 QPs — 328 image evaluations.
- **Compression quality:** Average PSNR increases from 30.42 dB to 37.48 dB as BPP increases from 0.112 to 0.698.
- **Runtime:** Encoding and decoding latency are recorded for each image and QP.
- **Energy:** Approximately 5,159.7 J for the complete evaluation run, estimated from sampled GPU power.
- **Memory:** Peak sampled GPU memory usage was 1,554 MiB; average sampled usage was 1,424.7 MiB.

kodak

https://r0k.us/graphics/kodak/

- **Dataset:** 24 Kodak images evaluated at 8 QPs — 192 image evaluations.
- **Compression quality:** Average PSNR increases from 27.8 dB to 36.32 dB as BPP increases from 0.297 to 1.25
- **Runtime:** Encoding and decoding latency are recorded for each image and QP.
- **Energy:** Approximately 309.4 J for the complete evaluation run.
- **Memory:** Peak sampled GPU memory usage was 383 MiB; average sampled usage was 335.4 MiB.

CLOSE

Pulse-MSE: References & Resources

MODEL & TRAINING DATA

- **Model:** Paper · Official code ·models
- **Training sources:** Open Images, SAM-1B, Flickr2K, DIV2K, CLIC. The released recipe uses low- and high-resolution training stages.
- **Evaluation:** CLIC Professional validation — 41 images; Kodak — 24 images. Evaluate **QP 0–7**; CLIC is primary, Kodak is a sanity check.

RESEARCH & TOOLS

- **References:** HAWQ-V3 — hardware-aware mixed precision/ILP; NVIDIA Model Optimizer — quantization tools; PyTorch ONNX — model export.
- **Tools:** Python, PyTorch, paper's github repository, RTX 3050 Laptop GPU, nvidia CLI tools, JSON/CSV and Matplotlib.
- **Planned:** FP16/INT8 PTQ → component × QP sensitivity map → QAT if needed → custom ILP precision selection. Investigate ONNX/TensorRT feasibility for custom entropy operations.

WORKFLOW: Released checkpoint → FP32 baseline → precision sensitivity → mixed-precision optimization → deployment evaluation

CODE: my complete set-up is present on my github (forked from their github codebase of PULSE and we use this for project)

NOTE: *Energy and memory are sampled whole-run estimates. QAT, ILP and ONNX/TensorRT deployment remain planned work; HAWQ-V3 is a reference, not a claimed reproduction but we refer it for an inspiration*